Ingestion Epic

Technical & Process Documentation

Three-channel document ingestion - Email (MS Graph), SFTP and Web Portal - into one validated, deduplicated funnel that feeds the downstream OCR pipeline. Covers user stories ING-01 through ING-06.

Document	**Ingestion Epic - Technical Documentation**
Track / Owner	**Ingestion Epic - Ayush (Full-Stack Developer)**
Version	**1.0 (Demo-phase build)**
Date	**15 June 2026**
Classification	**Internal - Project Stakeholders**
Status	**Released for review**

Table of Contents

Table of Contents 2

1. Executive Summary 4

2. Purpose & Scope 4

2.1 Objective . . . 4

2.2 In scope . . . 4

2.3 Out of scope (owned by adjacent epics) . . . 4

3. Position in the Platform 5

4. Requirements Covered (User Stories) 5

5. Technology Stack 5

6. Solution Architecture 6

6.1 Profiles & transport selection . . . 7

6.2 Module composition . . . 8

7. Pre-Processing Pipeline (ING-01 / ING-05) 8

7.1 Validation steps . . . 9

7.2 Accepted formats & limits . . . 10

8. Channel: Automated Email (ING-02) 10

8.1 Folder routing & resilience . . . 11

9. Channel: SFTP File Drop (ING-03) 11

9.1 Guards & outcome decisioning . . . 12

10. Channel: Web Portal Upload (ING-04) 13

11. Metadata & Data Contract 13

12. API Reference 13

13. Security & Access Control 14

14. Configuration Reference 14

15. Resilience, Idempotency & Error Handling 15

16. Testing & Quality Gates (ING-06) 15

17. Deployment & Runtime 16

18. Non-Functional Requirements 16

19. Dependencies & Open Items 16

20. Requirements Traceability Matrix 17

21. Glossary 17

Appendix A. Repository Structure 17

1. Executive Summary

The **Ingestion Epic** is the entry point of the Martinrea Accounts Payable (AP) Automation Platform. Its mandate is to collect supplier invoices from every channel the business uses today - vendor e-mail, partner / plant SFTP drops, and direct web-portal uploads - and funnel them through a single, strictly validated, deduplicated pre-processing stage before handing each accepted document to the Data & Repository service for storage and onward OCR extraction.

The service is built in **NestJS (TypeScript)** using a clean ports-and-adapters design: the three channel services and the validation pipeline contain all business logic and depend only on small interfaces, while interchangeable adapters provide the concrete transports (filesystem for local/demo, Microsoft Graph / IMAP for e-mail, *ssh2-sftp-client* for SFTP, and an HTTP / OCI back-end for storage). A single environment switch, `INGESTION_PROFILE`, flips the whole system between a zero-infrastructure **local** mode and a credentialed **prod** mode.

At a glance

Channels: Email (ING-02) • SFTP (ING-03) • Web Portal (ING-04), all converging on one pre-processing funnel (ING-01 / ING-05).

Guarantees: magic-byte type validation, 10 MiB size cap, SHA-256 content de-duplication, full quarantine audit trail, and idempotent re-delivery.

Quality gate: end-to-end suite across all three channels (ING-06) runs in CI with enforced coverage thresholds.

This document records the technology stack, the architecture, the end-to-end process for each channel (with flowcharts), the data and API contracts, security model, configuration, resilience behaviour, testing strategy and deployment approach - everything a reviewer, operator or future maintainer needs to understand and run the ingestion service.

2. Purpose & Scope

2.1 Objective

Automate the collection of invoices from external sources into a centralized staging area for downstream OCR processing, eliminating paper-based and manual e-mail handling. Every document that enters the platform does so through this epic, which makes its validation rules the first line of data-quality defence for the entire AP system.

2.2 In scope

- A scalable NestJS ingestion back-end with a shared pre-processing & format-validation service.
- Automated e-mail ingestion from designated AP mailboxes via Microsoft Graph (production) or IMAP (demo).
- Automated SFTP polling of per-plant incoming folders with safe download / delete semantics.
- A secured REST portal-upload endpoint for AP clerks (multipart/form-data).
- Hand-off of validated documents - with stamped metadata - to the Data & Repository (Blob) service.
- Quarantine of every rejected document with a machine-readable reason code for audit.
- A health endpoint and an automated end-to-end test suite executed as a CI merge gate.

2.3 Out of scope (owned by adjacent epics)

- OCR / data extraction and confidence scoring - **AI & OCR Epic** (Abhay).
- Document persistence in Azure Blob, the PostgreSQL schema and the OCR queue trigger - **Data & Repository Epic** (Roshni).

- Identity provider, role definitions and the approval workflow / state machine - **Workflow & Approvals Epic** (Mohd Aman).
- Epicor CMS master-data sync and CFDI / SAT validation - **External Integrations Epic** (Manav & Eswar).

3. Position in the Platform

Ingestion is the upstream-most track. It produces the `RECEIVED` state for every invoice and hands off to Roshni's data layer, which in turn triggers OCR. The table below summarises the immediate integration boundaries.

Direction	Counterpart	Interface / contract
Downstream	Data & Repository (Roshni)	`BlobUploadClient.upload()` / `.quarantine()` - de-dup by content hash; triggers OCR queue on insert.
Downstream	AI & OCR (Abhay)	Consumes the queue Roshni publishes; ingestion forwards CFDI XML as well as PDFs/images.
Auth	Workflow & Approvals (Mohd Aman)	Keycloak realm, role names and JWT claim path consumed by the portal auth guard.
Metadata	External Integrations (Manav)	Plant-code stamping on SFTP files supports Epicor attribution.
Upstream	Frontend portal (Dhwaj)	Calls `POST /api/ingestion/upload`; receives a documentId + staging status.

4. Requirements Covered (User Stories)

The epic delivers six user stories. Each is implemented and exercised by the automated suite.

Story	Title	Summary of delivered behaviour
ING-01 / 05	Foundation & Pre-Processing	NestJS service; format + size validation; quarantine of failures; validated hand-off with metadata; `GET /ingestion/health`.
ING-02	Automated Email Ingestion	Scheduled mailbox polling; attachment extraction; folder routing (Processed / No-Attachment / Failed); retry on transient failure; no files written to disk.
ING-03	SFTP Ingestion	Scheduled per-plant polling; partial-write grace; download, validate, then delete-on-success; retry vs. quarantine decisioning.
ING-04	Web Portal Upload	`POST /api/ingestion/upload` (multipart); JWT-guarded; mirrors core size/type rules; 201 with documentId + status.
ING-06	End-to-End Testing	Integration suite across all three channels (valid / oversized / invalid-type / duplicate); green-red CI gate; coverage artifact archived.

5. Technology Stack

The stack aligns with the platform-wide standard (NestJS / TypeScript back-end, Azure cloud, Keycloak identity, Docker / Kubernetes, GitHub Actions CI). The table groups the libraries actually used by the ingestion service.

Layer	Technology	Role in the ingestion service
Runtime	Node.js 20 LTS (Alpine container)	Service host; container base image `node:20-alpine`.
Language	TypeScript 5.7	Strongly-typed source across services, adapters and tests.
Framework	NestJS 10 (common / core / platform-express)	Dependency injection, module wiring, controllers, providers.

Layer	Technology	Role in the ingestion service
HTTP	Express (via platform-express) + Multer	Portal upload endpoint; multipart parsing with a hard size limit.
Scheduling	@nestjs/schedule + cron	Registers the e-mail and SFTP poll cron jobs.
Config	@nestjs/config	Loads `.env` and resolves profile / transport selection.
Validation	class-validator, class-transformer	DTO validation + global `ValidationPipe` (whitelist, transform).
Type sniffing	file-type 16.5.4	Magic-byte MIME detection (defeats renamed-extension attacks).
Hashing	Node.js crypto (SHA-256)	Content-hash de-duplication key.
Email (prod)	@microsoft/microsoft-graph-client, @azure/identity	App-only Graph access to M365 AP mailboxes.
Email (demo)	imapflow, mailparser	Dummy Gmail inbox over IMAP for the demo phase.
SFTP	ssh2-sftp-client	Real SFTP transport (connect-per-poll).
Auth	jose (JWKS verify) + Keycloak IdP	Cryptographic bearer-JWT verification on the portal endpoint.
Storage hand-off	Backend HTTP document API / OCI PAR / Azure Blob	Pluggable `BlobUploadClient` adapters.
Reactive utils	rxjs	NestJS framework dependency.
Testing	Jest, ts-jest, supertest	Unit + end-to-end channel suite with coverage thresholds.
Quality	ESLint, Prettier	Lint + format gates (run in CI).
Build	Nest CLI / tsc, ts-node	Compilation and on-demand ops scripts.
Container / CI	Docker, GitHub Actions	Image build; lint -> build -> unit -> e2e pipeline.

> **Platform target (per PRD)**
>
> Cloud-native deployment on **Docker + Kubernetes**; secrets in **Azure Key Vault**; documents in **Azure Blob Storage**; identity through **Keycloak**; delivery via **GitHub Actions + Argo CD** across DEV / QA / UAT / PROD.

6. Solution Architecture

The service follows a **ports-and-adapters (hexagonal)** pattern. Three thin channel components capture documents and converge on one pre-processing funnel; that funnel depends only on a small set of interfaces (the *ports*), and concrete *adapters* supply the transports. As a result, the same business logic runs unchanged whether the backing store is the local filesystem or a cloud API.

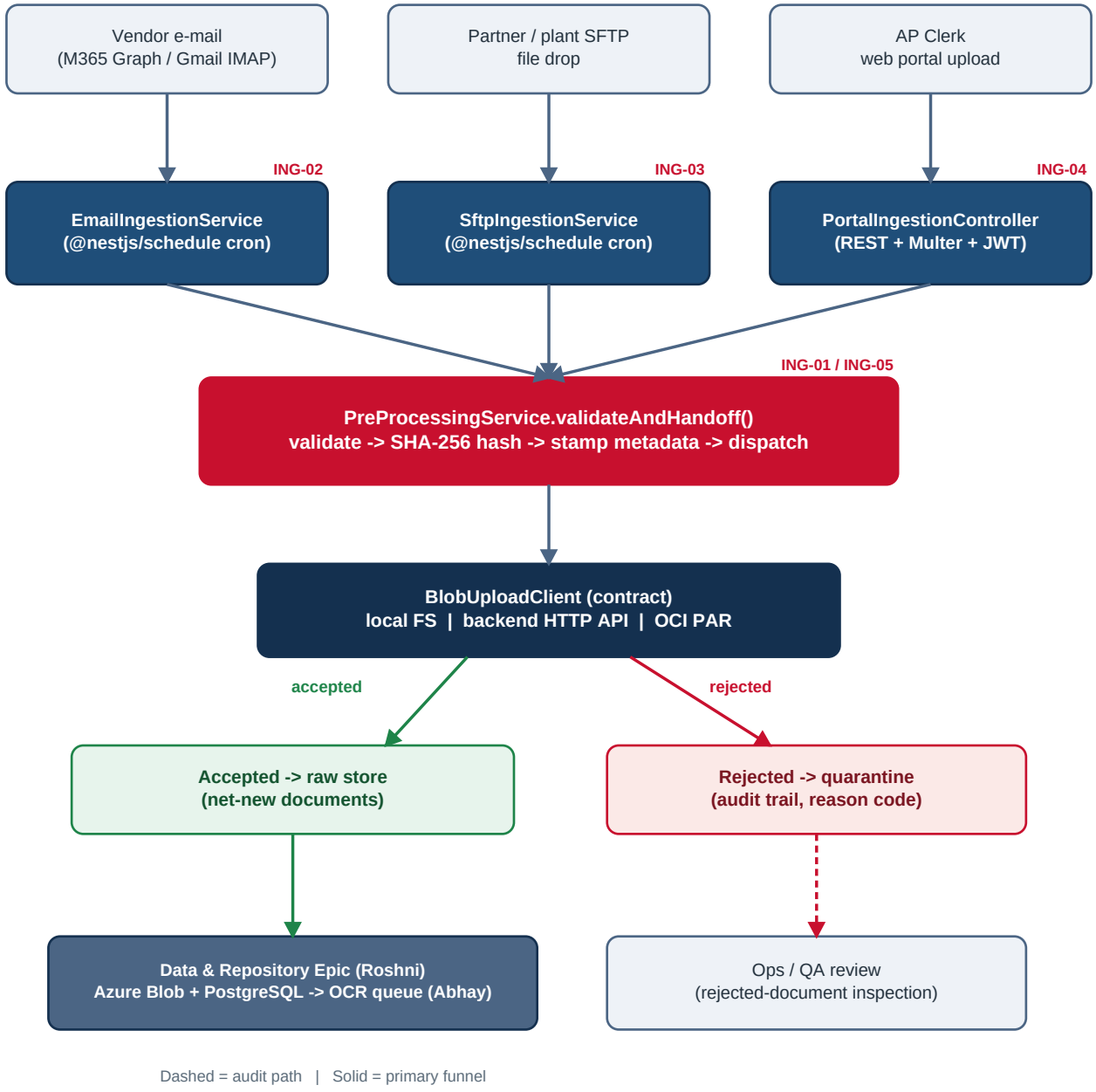

Figure 1 - Three-channel ingestion architecture. All channels converge on one validation funnel; accepted documents flow to storage and OCR, rejections to an audited quarantine.

6.1 Profiles & transport selection

`INGESTION_PROFILE` sets the defaults; each channel transport can also be overridden independently so a demo can mix real and local transports (for example a real Gmail inbox with local file storage). Non-local adapters validate their own credentials at construction and **fail loudly at boot** - a misconfigured deployment never silently drops invoices.

Injection token	Env override	Options (first = local default)
`MAIL_CLIENT`	`MAIL_TRANSPORT`	`local` (FS) • `imap` (Gmail demo) • `graph` (M365)
`SFTP_CLIENT_FACTORY`	`SFTP_TRANSPORT`	`local` (FS) • `ssh2` (real SFTP)
`BLOB_UPLOAD_CLIENT`	`BLOB_TRANSPORT`	`local` (FS) • `http` (backend doc API) • `par` (OCI bucket)

The interfaces in `src/ingestion/shared/` (`MailClient`, `SftpClient`/`SftpClientFactory`, `BlobUploadClient`) are the only contract the channel services know about; they are protocol-agnostic by design.

6.2 Module composition

- **main.ts** - bootstraps Nest, enables a global `ValidationPipe` and env-driven CORS, and listens on `PORT` (default 3002).
- **IngestionModule** - profile-aware factory providers wire each port to its local or prod adapter and log the chosen implementation at boot.
- **PreProcessingService** - the single validation funnel shared by all channels.
- **EmailIngestionService / SftpIngestionService** - schedule-driven pollers.
- **PortalIngestionController / HealthController** - the two HTTP surfaces.
- **KeycloakAuthGuard** - bearer-JWT guard protecting the portal endpoint.

7. Pre-Processing Pipeline (ING-01 / ING-05)

Every channel ends by calling `PreProcessingService.validateAndHandoff(buffer, meta)`. This is the **only** place validation lives, which keeps the rules consistent across all three entry points. The pipeline is intentionally fail-fast and audit-complete.

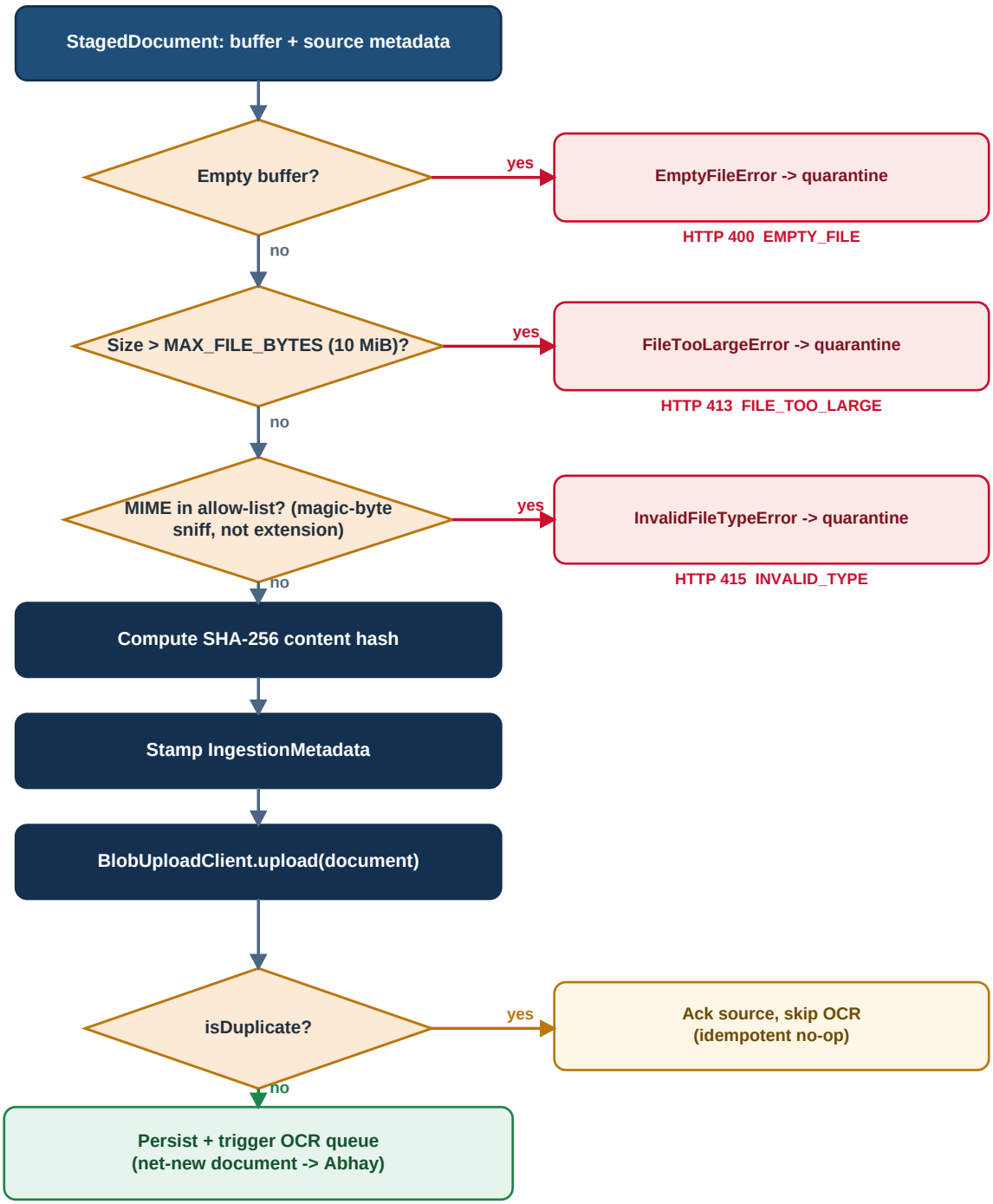

Figure 2 - Pre-processing validation funnel. Each gate that rejects a document ships the bytes to quarantine with a reason code before raising a typed error.

7.1 Validation steps

#	Step	Behaviour on failure
1	Empty-buffer check	`EmptyFileError` - HTTP 400; quarantined as `EMPTY_FILE`.
2	Size check vs. `MAX_FILE_BYTES` (10 MiB)	`FileTooLargeError` - HTTP 413; quarantined as `FILE_TOO_LARGE`.

#	Step	Behaviour on failure
3	Magic-byte MIME sniff vs. allow-list	`InvalidFileTypeError` - HTTP 415; quarantined as `INVALID_TYPE`. Catches renamed binaries (e.g. `evil.exe` → `invoice.pdf`).
4	SHA-256 content hash	Computed as the de-duplication key.
5	Metadata stamping	Builds the `IngestionMetadata` record.
6	Blob hand-off	`upload()`; if `isDuplicate` the source is still acked but OCR is skipped.

Type detection reads *magic bytes*, never the client-supplied extension. A small fallback recognises XML / CFDI payloads (which the sniffer does not always detect for tiny files) so Mexican `Comprobante` XML invoices are accepted rather than quarantined.

7.2 Accepted formats & limits

Policy	Value	Notes
Allowed MIME types	PDF, JPEG, PNG, TIFF, XML	PRD baseline (PDF/JPG/PNG/TIF) plus `application/xml` & `text/xml` for CFDI.
Maximum file size	10 MiB (10,485,760 bytes)	Enforced at Multer level (portal) and again in the funnel (all channels).
De-duplication	SHA-256 of file content	Identical bytes from any channel are treated as a successful no-op.
Quarantine reasons	EMPTY_FILE, FILE_TOO_LARGE, INVALID_TYPE, UNREADABLE	Stored alongside the rejected bytes for QA / ops inspection.

> **Design rule**
>
> Quarantine is **best-effort and never throws** - a failure to quarantine is logged but does not mask the original validation error. Validation is centralised: no channel duplicates type or size checks.

8. Channel: Automated Email (ING-02)

`EmailIngestionService` polls one or more AP mailboxes on a cron schedule (`EMAIL_POLL_CRON`, default every 2 minutes). For each unread message it streams every attachment buffer through the pre-processing funnel and then routes the message to a result folder. In production it authenticates to Microsoft 365 through Graph using the OAuth 2.0 client-credentials (app-only) flow; in the demo phase it reads a dummy Gmail inbox over IMAP. Attachment buffers are passed directly downstream - **no file is ever written to local disk**.

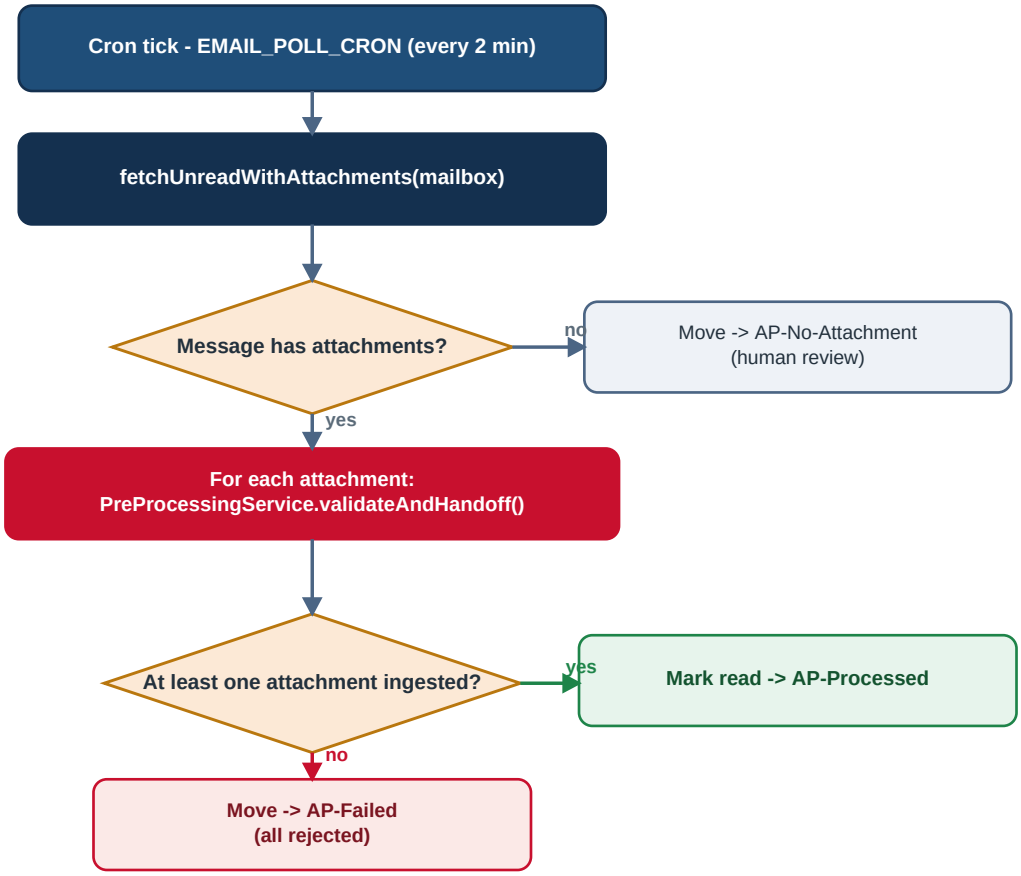

Figure 3 - E-mail polling and folder-routing flow. A message is only marked read and moved once its attachments have been processed.

8.1 Folder routing & resilience

Outcome	Destination folder	Rationale
≥1 attachment ingested	`AP-Processed`	Message acknowledged; marked read.
No attachments at all	`AP-No-Attachment`	Surfaced for human review (vendor terms may be in the body).
All attachments rejected	`AP-Failed`	Bytes already quarantined by pre-processing.

- **Fetch failure** (network / auth) is logged and the tick is abandoned; the next cron tick retries because the messages are still unread - nothing is lost.
- **Folder move** retries up to 3 times with exponential back-off; if it still fails the message stays unread for the next poll.
- **Idempotency:** re-polling the same message produces no duplicates because the content hash is de-duplicated downstream.

9. Channel: SFTP File Drop (ING-03)

`SftpIngestionService` polls each configured incoming path (`SFTP_INCOMING_PATHS`, one folder per plant) on the `SFTP_POLL_CRON` schedule. A fresh connection is opened per poll - long-lived SSH sessions die silently behind corporate firewalls - and is always torn down in a `finally` block.

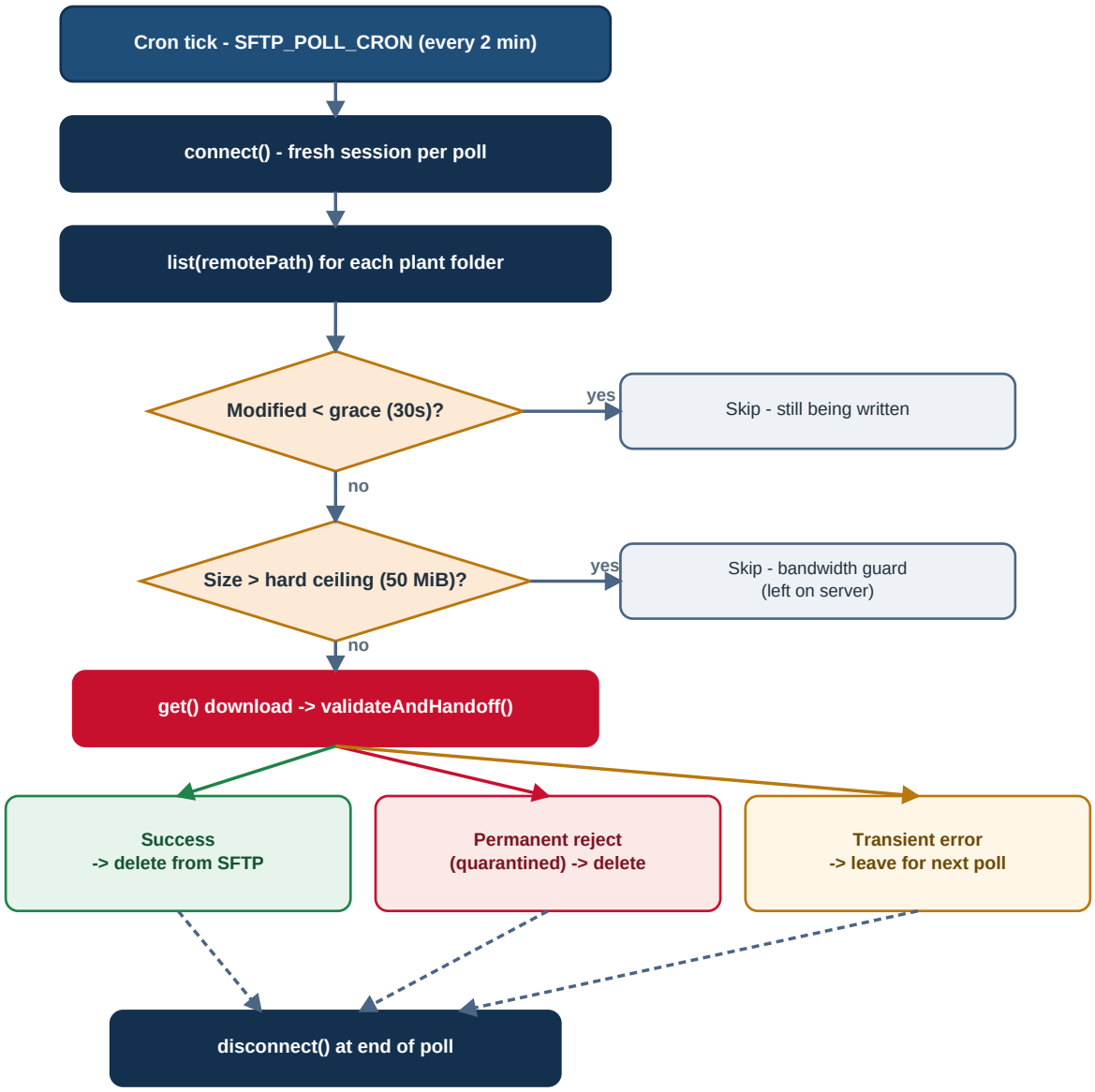

Figure 4 - SFTP poll flow with partial-write grace, bandwidth guard and the success / permanent-reject / transient-error outcome split.

9.1 Guards & outcome decisioning

Mechanism	Default	Purpose
Partial-write grace	30,000 ms	Skip files modified within the window - the scanner / partner may still be writing.
Bandwidth hard ceiling	50 MiB	Skip (without downloading) pathologically large files; ~5× the 10 MiB policy so genuine oversize invoices are still audited.
Plant-code inference	folder name	`/incoming/welland/` → `WELLAND` metadata for Epicor attribution.

Result of validateAndHandoff()	Action on SFTP source
Success	Delete from the server (prevents re-ingestion).
Permanent rejection (typed `IngestionError`)	Already quarantined - delete from the server so the same bytes are not re-quarantined every poll.

Result of validateAndHandoff()	Action on SFTP source
Transient failure (e.g. blob backend down)	Leave in place; the next poll retries.

10. Channel: Web Portal Upload (ING-04)

`PortalIngestionController` exposes `POST /api/ingestion/upload` accepting `multipart/form-data` (field `file`, plus optional `vendorHint` and `notes`). The endpoint is synchronous: it validates, hands off, and returns immediately with a `documentId` so the frontend can confirm the upload - it does not block on OCR, which runs asynchronously after Roshni's queue trigger.

- Protected by `KeycloakAuthGuard` - a valid bearer JWT is required (`AP_Clerk` or higher).
- Multer enforces the size cap *before* the handler runs; the pre-processing funnel re-checks at runtime (belt-and-braces) - both limits read the same `MAX_FILE_BYTES`.
- The same validation funnel and de-dup as the other channels; a duplicate returns status `DUPLICATE` rather than creating a second record.
- Captures the authenticated user (id / email) as source metadata for audit.

Success response (HTTP 201):

```
{
  "success": true,
  "data": {
    "documentId": "87c03cd8-1a9a-4eae-9f91-c4835820054d",
    "status": "STAGED",            // or "DUPLICATE"
    "estimatedOcrTimeSec": 120
  }
}
```

11. Metadata & Data Contract

Every accepted document is stamped with a sanitised `IngestionMetadata` record before it leaves the epic. This is the contract consumed by the Data & Repository service.

Field	Type	Description
sourceChannel	'email' | 'sftp' | 'portal'	Channel the document arrived through.
originalName	string	Original filename as received.
mimeType	string	MIME detected from magic bytes.
sizeBytes	number	Validated byte length.
contentHash	string	SHA-256 hex digest - the de-duplication key.
ingestedAt	string (ISO-8601)	Timestamp the document was accepted.
sourceMeta	object	Channel-specific context (mailbox / message id, remote path / plant code, uploader id, vendor hint, etc.).

The blob client returns an `UploadResult` (`documentId`, `blobPath`, `isDuplicate`). When `isDuplicate` is true, the ingestion service treats it as a successful no-op and skips the OCR hand-off.

12. API Reference

Method & path	Auth	Request	Success
`GET /ingestion/health`	None	-	200 `{ status:'ok', service, uptimeSec, time }`
`POST /api/ingestion/upload`	Bearer JWT	`multipart/form-data`: `file`, `vendorHint?`, `notes?`	201 `{ success, data:{ documentId, status, estimatedOcrTimeSec } }`

The e-mail and SFTP channels have **no HTTP trigger** - they run purely on `@nestjs/schedule` cron jobs. Operators can force a single poll on demand via the `poll:email` / `poll:sftp` scripts.

Error	HTTP	Code
Empty file	400	`EMPTY_FILE`
File too large	413	`FILE_TOO_LARGE`
Unsupported / renamed type	415	`INVALID_FILE_TYPE`
Missing / invalid bearer token	401	`Unauthorized`

13. Security & Access Control

The portal endpoint is protected by `KeycloakAuthGuard`, which supports three modes selected by `INGESTION_AUTH_MODE` (falling back to `INGESTION_PROFILE`). The guard always **fails closed** in production: a missing or invalid token - or missing configuration - returns 401.

Mode	Verification	When used
`local`	Dev bypass - any bearer accepted; injects a dev `AP_Clerk` (role overridable via `X-Dev-Role`).	Local dev, demos, the E2E suite.
`jwt`	Verifies an HS256 token signed with the shared `JWT_SECRET` issued by the workflow service.	Integrated stack without Keycloak.
`keycloak`	Cryptographically verifies the JWT against the Keycloak JWKS (issuer + audience) and maps the role claim.	Production identity.

Recognised roles are `AP_Clerk`, `Plant_Manager`, `Finance_Director` and `Admin`; the upload endpoint requires `AP_Clerk` as a minimum. Broader platform controls (TLS 1.2+ in transit, AES-256 at rest, 8-hour token expiry, AP-malware scanning via Defender for Storage post-upload) are inherited from the platform NFRs.

14. Configuration Reference

Key environment variables (full list with comments in `.env.example`):

Variable	Default	Purpose
INGESTION_PROFILE	local	Master switch: local (FS adapters) vs. prod (real adapters).
MAIL_TRANSPORT / BLOB_TRANSPORT / SFTP_TRANSPORT	(profile)	Per-channel transport overrides.
MAX_FILE_BYTES	10485760	Hard size cap (Multer + funnel).
ALLOWED_MIME_TYPES	PDF,JPEG,PNG,TIFF,XML	Comma-separated MIME allow-list (matched against magic bytes).
EMAIL_POLL_CRON / SFTP_POLL_CRON	0 */2 * * * *	Poll schedules (6-field cron, second granularity).

Variable	Default	Purpose
MAIL_AP_MAILBOXES	ap-canada@martinrea.com	Comma-separated mailboxes to poll.
IMAP_HOST / PORT / USER / PASSWORD	imap.gmail.com / 993	IMAP demo inbox credentials (Gmail app password).
GRAPH_TENANT_ID / CLIENT_ID / CLIENT_SECRET	-	Microsoft Graph app-only credentials (prod e-mail).
SFTP_HOST / PORT / USERNAME / PRIVATE_KEY	- / 22	Real SFTP connection (key sourced from Key Vault in prod).
SFTP_INCOMING_PATHS	/incoming/welland,/incoming/saltillo	Remote folders, one per plant.
SFTP_PARTIAL_WRITE_GRACE_MS	30000	Skip-window for files still being written.
SFTP_HARD_CEILING_BYTES	52428800	Bandwidth guard ceiling (50 MiB).
BLOB_API_BASE_URL / BLOB_API_AUTH_TOKEN	-	Backend document HTTP API (BLOB_TRANSPORT=http).
KEYCLOAK_ISSUER_URL / JWKS_URL / AUDIENCE / ROLE_CLAIM_PATH	-	Keycloak JWT verification settings.
PORT / CORS_ORIGINS	3002 / localhost:3000	HTTP listen port and allowed browser origins.

15. Resilience, Idempotency & Error Handling

- **Idempotent by content hash:** the same invoice arriving twice (re-poll, re-send, or via two channels) is de-duplicated downstream and surfaces as a no-op / `DUPLICATE`.
- **Transient vs. permanent failures:** permanent rejections (type / size / empty) are quarantined and the source is cleaned up; transient failures (backend down, network blip) leave the source in place for the next poll to retry.
- **Bounded retries:** e-mail folder moves retry 3× with exponential back-off; the platform standard is retry-with-back-off for all external calls.
- **Fail-closed boot:** prod adapters validate credentials at construction and abort start-up if misconfigured, so invoices are never silently dropped.
- **Connect-per-poll SFTP:** avoids dead long-lived sessions behind firewalls; disconnect always runs in `finally`.
- **Typed errors:** `IngestionError` carries an HTTP status, a machine code and a quarantine reason for consistent handling and audit.

16. Testing & Quality Gates (ING-06)

Quality is enforced automatically. Unit tests cover the services and the auth guard; the end-to-end suite boots the real `IngestionModule` under the local profile against throwaway temp directories and exercises every channel with the four ING-06 cases - valid, oversized, invalid-type and duplicate - plus health and auth checks.

Channel	E2E cases covered
Email (ING-02)	Valid → AP-Processed; no-attachment → AP-No-Attachment; bad type → quarantine + AP-Failed; duplicate idempotency.

Channel	E2E cases covered
SFTP (ING-03)	Valid (download + delete); oversized → quarantine; bad type → quarantine; duplicate no-op; CFDI XML accepted.
Portal (ING-04)	201 valid; 413 oversized; 415 + quarantine bad type; DUPLICATE on resend; 400 no file; 401 no token; PNG accepted.

Coverage gate: the Jest config enforces 70% branches and 80% lines / functions / statements across `src/**` (network-bound prod adapters are excluded - they are integration-tested against live infrastructure). The CI pipeline runs on every push and pull request to `main`:

Stage	Command	Gate
Lint	`npm run lint`	ESLint - no errors.
Build	`npm run build`	TypeScript compiles.
Unit + coverage	`npm run test:cov`	Thresholds enforced.
End-to-end	`npm run test:e2e`	All channel cases pass.
Artifact	upload `coverage/`	Report archived (14-day retention).

17. Deployment & Runtime

- **Container:** multi-step `node:20-alpine` image - install, copy, `npm run build`, expose 3002, run `node dist/main.js`.
- **Local / demo:** `npm run start:dev` boots in local profile with zero external infrastructure; `seed:local` + `poll:email` / `poll:sftp` exercise the full pipeline.
- **Production:** set `INGESTION_PROFILE=prod`, supply credentials, `npm run build` then `start:prod`; orchestrated under Kubernetes with secrets from Azure Key Vault.
- **Port map:** ingestion 3002, workflow 3001, integrations 3003, frontend 3000 - so all services coexist locally.

On boot the module logs the resolved adapter for each port (e.g. `MAIL_CLIENT -> GraphMailClient`) and registers the cron jobs, giving operators an immediate, unambiguous record of the active configuration.

18. Non-Functional Requirements

Category	Target relevant to ingestion
Throughput	Sized for ~450,000 documents/year (~1,233/day peak); horizontally scalable poller / API.
Performance	Portal upload returns synchronously; OCR runs async (est. ~120 s) so the UI is never blocked.
Security	TLS 1.2+ in transit, AES-256 at rest, JWT on the portal endpoint, secrets in Key Vault.
Reliability	Retry-with-back-off on external calls; idempotent re-delivery; fail-closed boot.
Compliance	Every rejection quarantined with a reason; CFDI XML accepted for SAT-bound flows; 7-year retention (platform).
Observability	Structured boot + per-document logs; services report to Datadog at platform level.

19. Dependencies & Open Items

The local build runs fully without any of the items below; they are the inputs required to flip individual channels to production. Resolved product decisions (12 Jun 2026) are already baked into the build.

Source	Outstanding input
Data & Repository (Roshni)	Final REST contract for blob upload / quarantine, service-to-service auth model, metadata schema confirmation, OCR-queue trigger ownership.
IT / DevOps	Azure AD tenant + Graph app registration, real SFTP host + service key, Key Vault URLs, DEV/QA/UAT/PROD environments and CI/CD wiring.
Workflow (Mohd Aman)	Keycloak realm / client config, confirmed role names and JWT claim path.
AI & OCR (Abhay)	Confirmation that the OCR consumer reads Roshni's queue; CFDI XML handling expectation.
QA (Jack H.)	Representative anonymised test-data set; load-test target; negative-path contracts.

> **Resolved decisions (frozen)**
>
> File-size cap stays at **10 MiB**; allowed types **PDF / JPG / PNG / TIF + XML** only; poll interval **every 2 minutes**; cross-channel duplicates use **silent de-dup**.

20. Requirements Traceability Matrix

Story	Acceptance theme	Implementing component	Verified by
ING-01/05	Validation, quarantine, health	PreProcessingService, HealthController	Unit + E2E
ING-02	Email poll, folder routing, no-disk	EmailIngestionService, Mail adapters	Unit + E2E
ING-03	SFTP poll, delete-on-success, retry	SftpIngestionService, SFTP adapters	Unit + E2E
ING-04	Portal upload, JWT, 201 + id	PortalIngestionController, KeycloakAuthGuard	Unit + E2E
ING-06	All-channel suite, CI gate, artifact	test/e2e + GitHub Actions	CI pipeline

21. Glossary

Term	Meaning
Adapter / Port	Concrete transport implementation / the interface it satisfies (hexagonal architecture).
CFDI	Comprobante Fiscal Digital por Internet - Mexico's SAT-regulated electronic-invoice XML.
Magic bytes	Leading bytes of a file used to detect its true type, independent of the filename extension.
Quarantine	Isolated store for rejected documents, kept with a reason code as an audit record.
Idempotency	Re-processing identical content has no additional effect (here, via SHA-256 de-dup).
Profile	`local` vs. `prod` adapter selection driven by `INGESTION_PROFILE`.
JWKS	JSON Web Key Set - the public keys used to verify Keycloak-issued JWTs.

Appendix A. Repository Structure

```
src/
  main.ts                          Bootstrap (ValidationPipe, CORS, port)
  app.module.ts                    Root module
  ingestion/
    ingestion.module.ts            Profile-aware adapter wiring (local | prod)
    pre-processing/                Validation / hashing / dispatch funnel
    email/                         ING-02 automated e-mail poller
    sftp/                          ING-03 automated SFTP poller
    portal/                        ING-04 portal upload endpoint + DTO
    health/                        GET /ingestion/health
    auth/                          Keycloak bearer-JWT guard
    shared/                        Interfaces (ports): mail / sftp / blob, types, errors
    adapters/
      local/                       Filesystem transports (zero-infra)
      prod/                        Graph / IMAP / ssh2 / HTTP / OCI adapters
scripts/                           seed-local-fixtures, poll-email-once, poll-sftp-once
test/                              fixtures + e2e/ingestion.e2e-spec.ts (ING-06)
.github/workflows/ci.yml           lint -> build -> unit(+coverage) -> e2e gate
Dockerfile                         node:20-alpine runtime image
```

End of document. Prepared from the demo-phase ingestion-service source and the Phase 1 Product Requirements Document.